

Smart Scan Strategy — RL Lane Handoff

What the reinforcement-learning half of SIH26055 has to build, what is already decided for you, and what three people need to learn to get through it without getting stuck.

Project: SIH26055 (DRDO) — *Smart Scan Strategy for Electronic Warfare*. Written 2026-09-01.

Read alongside, and outranked by: `docs/project/SIH26055_PROBLEM_STATEMENT.md`, `ENVIRONMENT_SPEC.md`, `EVALUATION.md`, `DECISIONS.md`.

Provenance note. Every number in this document is *quoted* from those files, where it was measured against the Turing HDF5 data and recorded with its evidence on the date given. Nothing here was re-measured for this handoff, except the test count in §2, which was re-run while writing it. If a number matters to a decision you are making, re-run the measurement rather than trusting this page.

- | | |
|--|--------------------------------------|
| 1 The problem, in one page | 9 Reward design — the real work |
| 2 Where the project is right now | 10 Algorithm choice |
| 3 Splitting the work across three people | 11 Training protocol |
| 4 The line you must not cross | 12 Evaluation and the two traps |
| 5 What you actually control | 13 Reading list, ordered |
| 6 The environment you will be training against | 14 Milestones and definition of done |
| 7 RL concepts you need, in the order you need them | 15 Working rules |
| 8 Mapping this problem onto RL | 16 Glossary |

1. The problem, in one page

A military radar receiver has to watch 18 GHz of spectrum. It cannot watch all of it at once — its instantaneous bandwidth is about one-eighteenth of the total. So it *sweeps*: it tunes to one 1 GHz-wide band, listens for a short **dwell** (50 or 100 ms), then moves to the next band. Meanwhile, hostile emitters out there are transmitting — but not continuously. A radar with a rotating antenna only points at you for a fraction of each rotation. A frequency-agile radar hops between frequencies. So an emitter is only *interceptable* in certain (band, time) cells.

You catch an emitter only if your receiver happens to be tuned to the right band at the moment that emitter happens to be illuminating you. Miss the overlap, and you never knew it was there. This is why the problem statement calls it a **two-dimensional search problem**: frequency and time.

Today's systems sweep in a fixed order computed in advance from a library of known emitter types. That is **open-loop**. It has a specific failure: it spends the same time on an empty band as on a band full of new threats, because it is not listening to its own results. The problem statement asks for a scheduler that learns, from its own hits and misses, where to point next.

THE RL LANE'S JOB IN ONE SENTENCE

Learn a policy that picks the next band to tune to, using only the history of what this receiver has itself seen this episode, so that emitters are found *fast* and *a lot of their transmissions get captured* — beating both a round-robin sweep and a published adaptive algorithm on both counts at once.

Why this is genuinely hard (and it is not obvious until you measure it)

There are two objectives and they pull in opposite directions. The team measured this on all 47 training scenarios (recorded in `DECISIONS.md` D14, amended 2026-08-30):

SCHEDULER	INTERCEPTION RATIO (PULSES CAPTURED)	EMITTER COVERAGE	CENSORED MEAN TIME-TO-FIRST- INTERCEPT
Camp on the busiest band, never move	57.4%	30.4%	23.78 s
Round-robin sweep	5.5%	95.7%	9.72 s
Pulse-capture oracle (ceiling, not a baseline)	66.8%	75.1%	15.04 s

Camping on one band captures ten times more signal than sweeping — and finds less than a third of the emitters, taking two and a half times as long to find the ones it does find. Sweeping finds nearly everything and captures almost nothing. **No trivial strategy is good at both.** That gap is the whole problem, and it is where your agent has to live.

This is not a quirk of our dataset. Apfeld et al. found the same thing on different data: their random baseline had the *best* "radars detected at least once" score and the *worst* efficiency (`docs/reference/scheduling/paperSSPD (1).pdf`).

THE SINGLE MOST IMPORTANT CONSEQUENCE

If you optimise one number, you will get a degenerate agent that looks excellent and is operationally useless. **Never report interception ratio without censored intercept time and coverage beside it.** This rule is not a style preference — it is the reason the problem is interesting, and it is written into `EVALUATION.md` §4.

2. Where the project is right now

The environment is being built in four layers. Two exist; two do not yet.

LAYER	FILE	WHAT IT DOES	STATUS
L0 — scenario	<code>rfenv/scenario.py</code>	Turns Turing recordings into per-emitter contributions; builds scenarios by replay or by sampling	Built
L1 — truth	<code>rfenv/truth.py</code>	The <code>Z / S / C</code> grid: what is really out there, per band per 50 ms slot	Built
L2 — receiver	<code>rfenv/receiver.py</code>	A dwell: adds noise, compares to threshold, declares hit or miss	Not yet
L3 — agent interface	<code>rfenv/env.py</code>	The Gymnasium environment you will call <code>reset()</code> and <code>step()</code> on	Not yet

Verified while writing this document: `.venv/bin/python -m pytest -q` → 23 passed. `ls rfenv/` shows `constants.py`, `scenario.py`, `truth.py` and no `receiver.py` or `env.py`.

SO WHAT CAN YOU DO BEFORE ENV.PY EXISTS?

Most of your first two weeks, actually. In rough order:

- **Learn.** §7 and §13. This is not filler — an RL team that starts coding before it understands advantage estimation will spend three weeks debugging a working algorithm.
- **Build the baselines** (§12). They do not need a learned policy — they need only `env.step()`'s *signature*, which §6 gives you. Write them against a small fake env now, swap in the real one later. This is the best possible ramp-up: you learn the interface, the metrics, and the problem's shape by implementing the things you have to beat.
- **Build the training harness** — config files, seeding, logging, run directories. Boring, and it is what separates a team that can answer "which run produced this number?" from one that cannot.
- **Write the reward candidates** as standalone functions with a fixed signature, so they can be swapped without touching the agent (§9).

3. Splitting the work across three people

Three roles. They are not three equal piles of RL code — one of them is barely RL at all, and it is the one that decides whether the project has a defensible result. Names are placeholders; assign them yourselves and write the assignment down.

R1 — Agent and training **IMPLEMENTER**

Owens: the policy. Algorithm choice, network, hyperparameters, the training loop, checkpoints, and the answer to "why did this run not learn?"

Deliverables: a trainable agent that beats round-robin on both primary metrics; a trained checkpoint that can be re-loaded and evaluated deterministically; a short record of what was tried and what failed.

Must understand deeply: MDPs and POMDPs; policy gradients; what an advantage estimate is and why variance matters; on-policy vs off-policy; entropy and exploration; the standard failure modes (policy collapse, reward hacking, silent environment bugs that look like slow learning).

Depends on: R2 for the environment wrapper and reward modules; R3 for honest evaluation.

Rough time to competence: Sutton & Barto ch. 1, 3, 6, 13 + Spinning Up + the PPO paper $\approx$ one focused week, in parallel with building the harness.

R2 — Environment integration and experiment harness **IMPLEMENTER**

Owens: everything between `rfenv` and the learner. The observation vector (including any extension you propose), the reward modules, vectorised/parallel environments, seeding and reproducibility, run configuration, and the logging that produces the artefacts `EVALUATION.md` §8 requires.

Deliverables: a wrapper the agent can train against at speed; a config system where a run is fully described by one file plus a seed; episode logs and `metrics.json` written automatically for every run.

Must understand deeply: the Gymnasium API contract (what `reset`, `step`, `terminated` vs `truncated`, `info` actually mean); observation normalisation; vectorised envs and how seeding works across them; enough of the RF model (§6) to know when the environment is lying to the agent.

Depends on: the environment lane for `receiver.py` and `env.py`. **This is the person who talks to that lane** — one interface owner, not three.

Rough time to competence: Gymnasium + Stable-Baselines3 docs, especially the "RL Tips and Tricks" page $\approx$ two to three days.

R3 — Baselines, evaluation and evidence

SUPPORT / ANALYSIS

Owens: everything that decides whether the RL result is *believed*. The non-learning baselines, the metric implementations, the comparison tables and plots, the waterfall renders, and the written record of what was tried.

Deliverables: baselines 1–6 from the ladder in §12; one script that takes any scheduler (learned or not) and produces the full metric table on a fixed scenario set; the final comparison; the `DECISIONS.md` entries for every choice the RL lane made; the figures that go in the SIH presentation.

Must understand: the metric definitions in `EVALUATION.md` §4 exactly — including *why* per-dwell hit rate and uncensored intercept time are wrong; the two traps in §12; enough RL vocabulary to read training curves and call out a suspicious result.

Does not need: to implement a policy gradient. This role is deliberately lighter on RL theory and heavier on rigour.

Why this role matters more than it looks: a scheduler that "wins" on a metric nobody checked is worth nothing at a defence review. R3 is the person who catches the camper masquerading as a learned policy, and who can say — with a number — what the agent actually bought us. Also the natural owner of the pitch narrative: what we tried, what the data showed, what changed our mind.

OPEN OWNERSHIP QUESTION — SETTLE THIS IN WEEK 1

The baseline ladder is defined in `EVALUATION.md` §5 but has no owner recorded. This document proposes R3 owns baselines 1–6. Confirm with Aamir / the environment lane that they have not already claimed it, then write the answer into `DECISIONS.md`. Two people building round-robin twice is the cheapest possible waste.

Where the three of you must actually be in the same room

1. **Observation vector.** R2 proposes, R1 says whether it is learnable, R3 checks it contains no truth-side leakage. One meeting, one decision, logged.
2. **Reward candidates.** All three — R1 knows what is trainable, R3 knows what the metrics will punish. (§9.)
3. **Final comparison run.** Everyone agrees the protocol *before* the numbers exist, because agreeing after is how a result stops being credible.

4. The line you must not cross

The environment is **frozen** once its four validation gates pass. Frozen means: these values live in `rfenv/constants.py` and no RL result may move them.

FROZEN	VALUE	WHY YOU CANNOT TOUCH IT
Band geometry	36 bands, centres 250 MHz to 17 750 MHz on 500 MHz spacing, each ± 500 MHz wide (so adjacent bands overlap by half)	Read from the Turing receiver metadata; changing it makes our numbers incomparable to the dataset
Slot clock	50 ms slots, 600 slots = one 30 s episode	Turing's minimum dwell
Dwell lengths	Seven bands dwell 100 ms, the other 29 dwell 50 ms; a full sweep is 2.15 s	Turing's own schedule
Noise floor N_0	-120 dB	Anchored to the receiver's own fields
Noise sigma σ	3 dB	Our receiver's design parameter
Detection threshold γ	-111 dB ($= N_0 + 3\sigma$)	See the box below — this is the important one
Metric definitions	<code>EVALUATION.md</code> §4	A metric redefined mid-project is a metric that proves nothing
Scenario sampling distribution	N drawn from the empirical detectable-emitter count, 1 to 82	Keeps training difficulty matched to the data

WHY γ IS FROZEN, AND WHY P_d IS NOT YOURS TO IMPROVE

γ is the level at which the receiver declares "I heard something". Lower it and you hear more real signals — and more noise. That trade-off is the receiver's ROC curve, and it is a property of the *detector*, not of the schedule.

If γ were trainable alongside your agent, the optimiser would simply lower it to manufacture hits: apparent interception improves, false alarms explode, and the comparison becomes meaningless. So:

- **P_d and P_{fa} are identical for every scheduler**, yours included. At $\gamma = -111$ dB, measured over the 47 train configs: $P_d = 0.822$, $P_{fa} = 1.35 \times 10^{-3}$, sensitivity -107.2 dB (recorded in D23/D26, 2026-09-01).
- **Your agent cannot improve detection. It can only aim the detector at more of the right cells.** That is what the scheduler-level metrics measure, and it is the only thing you are being judged on.

If you ever find yourself wanting to change a frozen constant to make training work, that is a signal about your reward or your observation, not about the environment. Raise it; do not edit it. (D15, D21, D25.)

5. What you actually control

Short list, and it is deliberately short. Everything here is *open* and is the RL lane's call — but each choice gets written into `DECISIONS.md` with its reasoning, because a decision nobody recorded is a decision the next person re-litigates.

YOURS TO DECIDE	NOTES
The RL algorithm	Not fixed anywhere. See §10 for a recommendation.
Network architecture	Including whether the policy is recurrent (see §8 — this matters more than usual here).
The reward function	Explicitly a hyperparameter: define candidates, train under each, pick the winner on the PS's own metrics (D7). §9.
Observation extensions	The baseline observation is defined (§6). You may extend it, subject to the cold-start rule in §6.
Exploration strategy	Entropy bonus, epsilon schedules, curriculum over scenario difficulty.
Training schedule	How many scenarios, how sampled, how many steps, when to stop.

6. The environment you will be training against

Read `ENVIRONMENT_SPEC.md` for the full thing. This is the part you need in order to write code against it.

What is really out there: the truth grid

For one 30-second scenario the environment holds three arrays, each $36 \text{ bands} \times 600 \text{ slots}$:

ARRAY	MEANING	WHO SEES IT
<code>Z[b, t]</code>	Occupancy. Is any emitter transmitting into this band at this slot? Boolean, threshold-free. This is the problem statement's own "transmission or non-transmission".	Evaluator only
<code>S[b, t]</code>	Level. The peak received signal in dB, or the noise floor (-120 dB) where nothing is transmitting.	Environment internals
<code>C[b, t]</code>	Pulse count. How many pulses are in this cell — the numerator of interception ratio.	Evaluator only

These come from real Turing recordings, not from a physics model. An emitter's "contribution" is the set of cells it lit up in an actual 30 s recording; a scenario is a set of such contributions and the world is their element-wise maximum. There is a pool of 3,443 contributions over 1,913 distinct emitters, and a new scenario is a draw from that pool (D25).

What the agent sees: the detection channel

When the receiver dwells on band `b` at slot `t`, it does *not* get `Z`. It measures a noisy version of the level and thresholds it:

$$Y = 1 \quad \text{iff} \quad S[b, t] + n \geq \gamma, \quad n \sim \text{Normal}(0, \sigma = 3 \text{ dB}), \quad \gamma = -111 \text{ dB}$$

So a real transmission can be missed ($P_d = 0.822$, not 1) and an empty cell can produce a false alarm ($P_{fa} = 1.35 \times 10^{-3}$). **Your agent learns from `Y`, which is sometimes wrong.** That is the point — a real ESM

receiver's feedback is also sometimes wrong, and an agent that cannot tolerate it would not survive deployment.

The Gymnasium interface

```
action_space      = Discrete(36)          # which band to tune to next
observation_space = Box(shape=(109,))     # 36 x 3 + 1

[ 0: 36] per-band empirical hit rate      - how often looking there paid off
[ 36: 72] per-band visit density          - how much of my time went there
[ 72:108] per-band staleness              - how long since I last looked
[108]    normalised episode time          - how much of the 30 s is gone

reward           = pluggable (see §9)
info             = truth-side quantities: per-emitter first-intercept slots,
                  cell occupancy. FOR THE EVALUATOR ONLY. Never feed to the agent.
reset(seed, options={scenario})
                  = deterministic replay (validation) or a sampled draw (training)
```

Those three per-band quantities are not arbitrary — they are the three things a scheduler needs in order to estimate the three figures of merit the problem statement asks for: detection probability, intercept rate, and staleness/intercept time respectively.

COLD START — THE RULE THAT SHAPES EVERYTHING

Every episode begins with **zero** emitter knowledge. No threat library, no map of who transmits where, no carry-over from the previous scenario. The observation is built *only* from the agent's own scan history this episode.

This is not a simplification, it is the requirement: the problem statement's title condition is "in the absence of prior reliable intelligence of emitters and their operating characteristics" (D20). It is also what makes the explore/exploit tension real — with a prior library, camping would be defensible; without one, you must keep discovering.

Practical consequence: any observation extension you propose must be computable from the agent's own history. If it needs `Z`, `S`, `C`, emitter identities, or anything from `info`, it is leakage and the result is invalid. R3 should check every extension against this.

Two subtleties that will bite you if nobody flags them

SUBTLETY 1 — AN EPISODE DOES NOT HAVE A FIXED NUMBER OF STEPS

An action selects a band; the dwell then runs that *band's* native length — 100 ms (2 slots) for seven of the 36 bands, 50 ms (1 slot) for the other 29. So an episode of 600 slots is somewhere between 300 and 600 agent decisions depending on which bands were chosen.

This is a direct reading of `ENVIRONMENT_SPEC.md` §L2 and D3/D16, not an independent claim. It means: variable-length episodes; a bootstrapping/discounting decision (does a 100 ms dwell discount twice?); and a subtle incentive — the wide-dwell bands cost double, so the agent is implicitly choosing how to spend time, not just where to look. Decide how you handle it and write it down.

SUBTLETY 2 — REWARD ON Y OR REWARD ON Z ?

`ENVIRONMENT_SPEC.md` names "+1 per true hit" as the first reward candidate. "True hit" implies Z — truth-side. But the agent only *observes* Y . Both are defensible and they are genuinely different designs:

- **Reward on Z** : lower-variance training signal, no false-alarm noise in the gradient. Legitimate — the reward is a training-time construct and a simulator is allowed to be privileged. But it trains an agent against a cleaner world than it will observe.
- **Reward on Y** : matches deployment exactly (a real system is only ever rewarded by what it declares), and is the honest reading of the PS's "trained based on hits and misses". Noisier.

This is not settled anywhere. It is the RL lane's first real decision. Settle it early — it changes what your reward functions can even reference — and record it in `DECISIONS.md`.

7. RL concepts you need, in the order you need them

This is the list of things where, if you do not understand them, you will lose days. Everything else can be looked up when you hit it. R1 needs all of it; R2 needs 1–4 and 8; R3 needs 1, 2 and 9.

1. MDP: state, action, reward, transition, discount

The vocabulary everything else is written in. A policy π maps state to a distribution over actions; the return is the discounted sum of future rewards; the agent maximises expected return. Know what γ (the discount, unrelated to our detection threshold γ — sorry) does to an agent's horizon. In our problem an episode is 30 s and a discount that is too aggressive will produce an agent that cannot justify exploring.

2. POMDP: why our agent cannot see the state

The true state is the 36×600 grid of who is transmitting. The agent sees a 109-number summary of its own history. It is *partially observed*, and the observation is a hand-built belief summary. Two direct consequences: (a) the optimal policy may need memory beyond what those 109 numbers carry, which is the argument for a recurrent policy; (b) improving the observation can beat improving the algorithm, so R2's work is not plumbing.

3. Exploration vs exploitation — and why our version is unusually literal

In most RL problems this is a metaphor about the learning process. Here it is the actual task: the agent must physically choose between re-visiting a band it knows is productive and probing one it has not seen. The measured tension in §1 is explore/exploit. Read Sutton & Barto ch. 2 (bandits) even though we are not doing bandits — it is the cleanest statement of the trade-off you will ever read.

4. Restless multi-armed bandits — the framing worth knowing by name

Our problem is close to a textbook *restless bandit*: 36 arms, you pull one per step, and the arms keep changing whether you look at them or not (an emitter's beam rotates whether or not you are tuned to it). Knowing this gives you two things: intuition (staleness matters because unobserved arms drift), and pitch value (we can say what class of problem this is). We are not required to implement a Whittle index.

5. Value functions, advantage, and why variance is the enemy

$V(s)$ is expected return from a state; $A(s,a)$ is how much better action a is than the policy's average. Policy-gradient methods work by pushing up the probability of actions with positive advantage. Because our reward is sparse and noisy — a dwell either catches something or does not, and sometimes lies — **advantage estimation quality is the difference between learning and not learning**. Understand GAE and what its λ knob does.

6. On-policy vs off-policy

On-policy (PPO, A2C) learns from data the current policy just generated — stable, sample-hungry. Off-policy (DQN, SAC) reuses a replay buffer — sample-efficient, more fragile. Our environment is fast and simulated, so sample efficiency is cheap and stability is worth more. That is the argument for PPO in §10, and you should be able to reconstruct it yourself.

7. PPO specifically

The clipped surrogate objective, why clipping exists (stop a single update from destroying the policy), and the handful of hyperparameters that actually matter: learning rate, number of steps per update, number of epochs,

entropy coefficient, GAE λ . Do not tune all of them at once.

8. Vectorised environments, seeding and reproducibility

You will run many environments in parallel. Know how seeds propagate, why a seeded run must reproduce exactly, and why "it worked yesterday" is not a result. R2 owns this and it is worth doing properly on day one rather than retrofitting it after the first surprising number.

9. Reading a training curve, and the standard failure modes

Reward going up is not success — reward going up is your agent finding whatever your reward actually said, which is often not what you meant. Learn to recognise: *policy collapse* (entropy $\rightarrow 0$, agent picks one action forever — in our case, that is literally the camper); *reward hacking*; and the flat curve that means the environment is broken rather than the agent being slow. When a run looks too good, check whether the agent is camping. It usually is.

8. Mapping this problem onto RL

RL TERM	HERE
Episode	One 30 s scenario, 600 slots, one draw of emitters
Step	One dwell — 1 or 2 slots depending on the band chosen
Action	Which of the 36 bands to tune to next (<code>Discrete(36)</code>)
Observation	109 numbers summarising this episode's own scan history
Hidden state	The 36×600 truth grid — never observed
Reward	Derived from hits/misses; the exact form is yours to choose (§9)
Transition	Deterministic in time (the world advances regardless); stochastic in what you hear (the noise draw)
Termination	Fixed horizon: episode ends at 30 s. Nothing terminates early

The one structural thing to notice: the world evolves whether you look at it or not. A band you ignored for 10 s has been busy or quiet without telling you. This is what makes staleness a meaningful feature and what makes the problem restless rather than a plain bandit.

9. Reward design — the real work

The reward is explicitly a hyperparameter here (D7). You define candidates, train under each, and pick the winner using metrics the reward cannot influence — interception ratio and censored intercept time — because scoring a reward by numbers it generated itself is circular.

Build reward functions as swappable modules with a single fixed signature from day one. R2 owns the plumbing; all three of you own the candidate list.

Candidate A — dense hit reward (the stated default)

+1 per hit in the dwell. Simple, dense, easy to learn. **Expected failure mode: it produces the camper.**

Camping maximises hits per dwell and the measured tables in §1 show it scores 57.4% interception ratio doing so. Train it anyway — it is your reference point, and the fact that it degenerates is itself a result worth showing in the pitch.

Candidate B — discovery-weighted

Large bonus for the first intercept of a previously unseen emitter, small reward for repeat hits. Directly targets intercept time. Two problems to solve: the agent cannot identify emitters (deinterleaving is explicitly out of scope, D19), so "previously unseen" has to be defined from the environment's truth side at training time — which is legitimate but must be declared; and it may over-explore and lose interception ratio.

Candidate C — staleness / information-gain shaped

Reward hits, plus a term that grows with how long a band has gone unvisited. Cheap, computable purely from the agent's own history (so no truth-side dependence), and directly encodes the restless-bandit intuition. The risk is that it degenerates into round-robin if the staleness term dominates. The weighting is the experiment.

HOW TO JUDGE THEM — DECIDE THIS BEFORE YOU TRAIN ANYTHING

Every candidate is scored on the same fixed scenario set with the same seeds, and reported as the full scheduler-level table (§12), never as reward-per-episode. **Reward values are not comparable across reward families** — an agent that scores 400 under Candidate A and 90 under Candidate C has told you nothing. Only the PS metrics can rank them (D7).

10. Algorithm choice

Not fixed by any project document — `PROJECT_ARCHITECTURE.md` §6 deliberately leaves it open. What follows is a **recommendation with reasoning**, not a decision. Make the decision yourselves, then record it.

Recommended default: PPO, with a recurrent variant as the first upgrade. The reasoning:

- Discrete action space of 36 — the easiest case for policy gradients.
- The simulator is fast, so PPO's sample hunger costs wall-clock, not feasibility.
- PPO is stable and forgiving. You will be changing the reward repeatedly (§9), which means retraining often; an algorithm that needs per-reward hyperparameter surgery will eat the schedule.
- It is well-documented and widely implemented, so a bug is more likely to be yours than the library's — which is what you want when debugging.

Why recurrent (LSTM/GRU) is the natural second step: the problem is a POMDP (§7.2), and the 109-number observation is a hand-designed summary of history. A recurrent policy can learn its own summary. Try feed-forward first — if it plateaus below the heuristic baselines, recurrence is the first thing to reach for, ahead of more hyperparameter tuning.

Use a maintained implementation (Stable-Baselines3, or SB3-contrib for RecurrentPPO) rather than writing PPO from scratch. Writing it yourself is an excellent way to *learn* it and a poor way to ship it; if R1 wants to implement it once as a learning exercise, do that in a scratch directory, not in the project.

What to reject and why (so you do not re-litigate it): DQN is plausible on a discrete action space but is more fragile under sparse noisy rewards and needs a replay buffer whose contents go stale as the policy changes. Contextual-bandit approaches throw away the temporal structure that makes staleness matter. Neither is wrong — they are just not where a small team's first month should go.

11. Training protocol

Which scenarios you train on

PURPOSE	SCENARIOS
Environment validation (gates 1–4)	The 47 deterministic train replays — <i>not yours</i> , the environment lane owns this
RL training	Sampled draws from the emitter pool, train configs only. No episode should repeat
Scheduler comparison	One fixed seeded set of sampled scenarios, plus the 47 replays — identical for every scheduler including yours
Final claim	The 45 held-out test pairs, once , after the whole system is frozen

THE HELD-OUT DATA

45 scan/stare pairs under `data/turing/*/test_*/` are held out. Selection rule was fixed in advance, before any result was seen (D8). **They are touched exactly once, at the very end.** `scenario.py` refuses these paths unless an explicit `allow_heldout=True` is passed, and logs every such call — so touching them by accident is hard, and touching them on purpose leaves a trace. Do not tune against them, do not "just check", do not use them for early stopping. Every use gets recorded.

Why sampled scenarios rather than the 47 replays

47 scenarios × 600 slots is roughly 28,000 decision steps per pass. An agent trained on 47 fixed worlds will memorise them, and memorisation is indistinguishable from strategy right up until the held-out evaluation says otherwise. The scenario sampler exists precisely to prevent this: it draws N emitter contributions from the pool, N taken from the empirical distribution of detectable emitters per config (1 to 82), so difficulty stays realistic while episodes never repeat (D25).

What every run must produce

Non-negotiable, because `EVALUATION.md` §8 defines evaluation in terms of these artefacts:

1. **Episode log** — per slot: time, band chosen, dwell length, declared hit Y, true occupancy, pulse count, peak level.

2. **Emitter table** — per emitter: detectable activity interval, first/last intercept slot, intercept count, bands seen in.
3. **Waterfall render** — the 36×600 grid as a frequency-vs-time heatmap with the scheduler's path drawn over it and hits marked. *Look at these*. It is the fastest way to see that your agent is camping, oscillating, or ignoring half the spectrum — a training curve will not tell you that, and this is also the standard ESM operator view, so it doubles as a presentation asset.
4. **metrics.json** — one per run.

Reproducibility

A run is described by one config file plus a seed, and re-running that pair reproduces the numbers. Report multiple seeds with spread, never a single run. Report per-scenario distributions, not just a grand mean — scenario difficulty spans 2 to 99 emitters and a mean over that range hides almost everything.

12. Evaluation and the two traps

Metric definitions live in `EVALUATION.md` §4 and are frozen. Do not redefine one in your training code — call the shared implementation, so that the number in your TensorBoard and the number in the final table are the same number.

The table you must always report

METRIC	DEFINITION	DIRECTION
Interception ratio	Intercepted illuminations $\div$ total illuminations — a pulse counts as intercepted iff your tuned band window contained it at its time of arrival. Per illumination, not per dwell.	higher
Censored mean intercept time	Mean over detectable emitters of (first intercept – activity start), with an emitter never intercepted counted at the full 30 s	lower
Emitter coverage	Fraction of detectable emitters found at least once — supporting diagnostic, always printed beside the two above	higher
Average intercept rate	Distinct emitter-intercepts per second	higher
Average reward / cost	Your accumulated reward. Comparable only within one reward family; never used to rank across rewards	—

TRAP 1 — PER-DWELL HIT RATE IS MISLEADING

A camper scores 85–90% per-dwell and looks near-optimal while capturing 30% of the emitters. The per-illumination definition above is the honest one and is what the literature uses. If you see a per-dwell hit rate in a plot, it is a debugging aid, not a result.

TRAP 2 — UNCENSORED INTERCEPT TIME REWARDS NOT LOOKING

Averaged over only the emitters it *found*, the camper appears *faster* (6.5 s) than round-robin — because it only ever finds the loudest, easiest emitters and the ones it missed silently drop out of the average. Censoring at episode end reverses it correctly: 23.8 s vs 9.7 s. **An average over successes only is a measure of how little you attempted.**

The ladder you have to climb

#	BASELINE	WHAT IT TELLS YOU
1	Random	Lower bound; the coverage-heavy extreme
2	Round-robin	The open-loop strategy the PS targets. The floor to beat
3	Turing reference sweep	The dataset's own schedule — makes our numbers comparable to the recordings
4	Greedy static (camper)	The degenerate exploit, included precisely to show a single metric can be gamed

#	BASELINE	WHAT IT TELLS YOU
5	Recency / activity heuristic	Simple adaptive benchmark
6	Apfeld adaptive	Published non-learning adaptive strategy (docs/reference/scheduling/paperSSPD (1) .pdf §II). The serious bar
7	RL scheduler	Yours
—	Pulse-capture oracle	Ceiling. A reference line, not a competitor

Beating round-robin is the minimum. Beating Apfeld is the claim worth making. Implementing Apfeld's algorithm is R3's most demanding task and should start early — it is a published algorithm being re-implemented from a paper, which always takes longer than expected.

THE TARGET, QUANTIFIED

The camper wins interception ratio (57.4%) and loses intercept time (23.78 s). Round-robin wins intercept time (9.72 s) and loses ratio (5.5%). **Your agent's job is to Pareto-dominate that pair** — approach round-robin's intercept time while multiplying its interception ratio. That is the success condition, stated as a number, and it was fixed before you started training, which is what makes it honest.

13. Reading list, ordered

Read in this order. Stop when you can explain the concept to the person next to you; do not read the whole book.

Core RL — everyone, in week 1

- **OpenAI Spinning Up, "Introduction to RL", parts 1–3.** The fastest correct grounding that exists. Two hours. Start here, not with the textbook.
- **Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed.).** Ch. 1 (the problem), ch. 2 (bandits — read it for the explore/exploit statement), ch. 3 (MDPs). R1 additionally: ch. 6 (temporal-difference learning) and ch. 13 (policy gradients).
- **Spinning Up, "Kinds of RL Algorithms" and the PPO page.** Then Schulman et al., *Proximal Policy Optimization* (arXiv:1707.06347) — the paper is short and readable.

Practical — R1 and R2

- **Gymnasium documentation** — the API contract, spaces, wrappers, vector envs. R2 should read this properly rather than skimming.
- **Stable-Baselines3 documentation** — PPO, VecEnv, callbacks, evaluation helpers. And in particular the "**RL Tips and Tricks**" page, which is the single highest-value page on this list for a team shipping under a deadline.
- **SB3-contrib RecurrentPPO** — when you get to §10's second step.
- Optional, R1: Schulman et al., *Generalized Advantage Estimation* (arXiv:1506.02438).

Domain — in this repository, and worth more than more RL reading

- `docs/project/SIH26055_PROBLEM_STATEMENT.md` — the actual requirement. Everyone, first. It is short.
- `docs/project/ENVIRONMENT_SPEC.md` and `EVALUATION.md` — everyone. These are the contract you are building against.
- `docs/project/DECISIONS.md` — skim all of it once so you know what has already been settled; read D5, D7, D14, D15, D19, D20, D21, D25 closely.
- `docs/reference/scheduling/paperSSPD (1).pdf` (Apfeld et al.) — the adaptive strategy you must beat. R3 reads §II in detail.
- `docs/reference/scheduling/optimumsearch.pdf` (Köksal) — closed-form intercept time and probability of intercept for the periodic case. Useful for sanity-checking that a learned policy is not doing something impossible.
- `docs/reference/scheduling/Dynamic Scan Scheduling.pdf` (Dutertre) — states the open-loop limitation this project exists to remove; good pitch material.
- `docs/reference/background/SIH- Smart Scan Strategy.pdf` §3–5 — receiver sensitivity, P_d and P_{fa} as they are used in the EW literature, which is where the framing in §4 comes from.

IF A CONCEPT DOES NOT LAND

Most of the RF/EW vocabulary here is new to most of the team. If probability of intercept, or what a policy gradient *is*, is not clicking from the reading — that is a conceptual question, and it is faster to take it to a chat assistant and have it explained from scratch than to keep re-reading. Split concept-learning from implementation deliberately; do not try to learn RL by debugging RL.

14. Milestones and definition of done

#	MILESTONE	OWNER	DONE MEANS
M0	Everyone has read §1–6 and the problem statement	all	Each person can state the two objectives and why they conflict, without looking
M1	Baselines 1–5 implemented against the interface in §6	R3	They run on a stub env and produce the full metric table
M2	Training harness: config + seed + logging + the four artefacts	R2	A run is reproducible from its config file alone
M3	Reward modules A, B, C written and swappable	R2 + all	Swapping a reward changes one line of config
M4	Reward-on-Y vs reward-on-Z settled (§6, subtlety 2)	all → DECISIONS.md	Written down with reasoning
M5	<code>env.py</code> lands; wrapper integrated; a random policy trains without crashing	R2	End-to-end run completes and logs
M6	First learned agent beats round-robin on both primary metrics	R1	On the fixed comparison set, multiple seeds, spread reported
M7	Apfeld baseline implemented	R3	Reproduces the paper's qualitative behaviour
M8	Reward candidates compared; winner chosen on PS metrics	all	Full table for each candidate; choice recorded in DECISIONS.md
M9	Agent beats Apfeld — or we can say precisely why not	R1 + R3	The claim worth making, with the evidence behind it
M10	System frozen; held-out evaluation run once	all	Numbers recorded; the use of the held-out set logged

M6 is the point at which the project has an RL result at all. M9 is the point at which it has one worth presenting. Everything between M0 and M5 is scaffolding, and skipping it is the most reliable way to arrive at M6 with a number nobody can defend.

15. Working rules

These come from `CLAUDE.md` and the research protocol and apply to the RL lane exactly as to everyone else.

- **Every number is traceable.** When you quote a figure, say how you got it — which script, which run, which seed. "I measured it on run 2026-09-14-ppo-c-seed3" and "I remember it being about 40%" are different claims and only one of them belongs in a table.
- **A measurement from a previous session is not a fact.** Re-run it, or label it unverified.
- **Record every decision in `DECISIONS.md`**, including small ones, with the *why*. The reasoning is what makes a decision reusable; the decision alone is not. This is also where the SIH pitch narrative comes from — what we tried, what the data showed, what changed our mind.

- **Do not change anything on the freeze list** (§4). If an RL result seems to require it, the environment must be re-validated from gate 1 and every baseline re-run — which is a project-level decision, not a lane-level one.
- **Surface findings as you get them.** Not in an end-of-week summary. A finding that only ever existed in a chat log is a finding the project has lost.
- **Flag improvements-for-later separately** from what you are doing now, rather than silently fixing them (scope creep) or silently dropping them (lost work).

16. Glossary

TERM	MEANING HERE
Band	One of 36 tuning positions, each ± 500 MHz around its centre. Adjacent bands overlap by half
Dwell	One period of listening on one band: 50 ms (29 bands) or 100 ms (7 bands)
Slot	The 50 ms time quantum. 600 slots per 30 s episode
Sweep	One pass through all 36 bands in order: 2.15 s
Illumination / pulse	A single transmission from an emitter that reaches our receiver
Intercept	We were tuned to a band whose window contained a pulse, at the time it arrived
Interception ratio	Fraction of all illuminations we captured. Per pulse, not per dwell
Intercept time (TTI)	Time from an emitter becoming detectable to our first intercept of it. Censored at 30 s if never found
Coverage	Fraction of detectable emitters found at least once
P_d	Probability of declaring a hit given the cell really was occupied. A receiver property; the same for every scheduler
P_{fa}	Probability of declaring a hit given the cell was empty. Also a receiver property
Sensitivity	The signal level at which P_d reaches a stated value at the operating P_{fa} . Quoted with both, never alone
γ (gamma)	The detection threshold, -111 dB. Frozen. Not the RL discount factor, which unfortunately shares the letter
Z, S, C	Truth arrays: occupancy (boolean), level (dB), pulse count. 36×600 each
Y	What the receiver <i>declares</i> when it looks. Sometimes wrong. This is what the agent learns from
Cold start	Every episode begins with zero prior emitter knowledge. Required by the problem statement
ESM	Electronic Support Measures — the passive listening half of electronic warfare
Deinterleaving	Separating a mixed pulse stream by emitter. The Turing dataset's native task; explicitly <i>not</i> ours (D19)

Where this document is wrong, the source wins. The problem statement outranks `ENVIRONMENT_SPEC.md` and `EVALUATION.md` , which outrank this handoff. If you find a disagreement, it is a finding — report it rather than picking one.